

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 3– ASSIGNMENT**

Course Code:	CSA0901	Course Title:	Programming in Java ASSESSMENTTOOL3 - ASSIGNMENT
CO Assessed:	CO3 – Precision (BL3,BL4)	Assessment:	
Weightage:		Total Marks:	100
CO1 Statement:	Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques		
Student Name:	S.Aarathana	Reg. No.:	1921260021
Section Batch: /	SLOT B	Date:	13/08/2026

Code of Conduct

I S.Aarathana (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 10 questions Total: 100 Marks.
- No negative marking. Un answered questions carry zero marks.
- No calculators or electronic devices permitted.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem	20%	Identifies all	Minor missing	Partial	Incorrect

Understanding & Context Analysis		inputs, outputs, constraints accurately	elements	understanding	interpretation
Application of Concepts	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Analytical & Decision-Making Skills	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Solution Design & Approach	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Clarity, Justification & Presentation	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Try, Catch and Finally

Question: Write a Java program to demonstrate the use of try, catch, and finally blocks for handling multiple built-in exceptions.

Program

```
class ExceptionDemo {
    public static void main(String[] args) {
        try{
            int a= 10, b = 0;
            System.out.println(a / b);
        }
        catch(ArithmeticException e) {
            System.out.println("Arithmetic Exception: Cannot divide by zero");
        }
        catch(ArrayIndexOutOfBoundsException e) {
            System.out.println("Array Exception: Invalid index");
        }
        finally {
            System.out.println("Finally block executed");
        }
    }
}
```

Output

```
Arithmetic Exception: Cannot divide by zero
Finally block executed
```

Concepts Used

- try – contains code that may cause an exception.
- catch – handles specific exceptions.
- finally – executes whether an exception occurs or not.
- Multiple catch blocks demonstrate handling different built-in exceptions.

2. User-Defined InvalidAgeException

Question: Write a Java program to create a user-defined InvalidAgeException and check whether a person is eligible to vote.

Program

```
class InvalidAgeException extends Exception {
    InvalidAgeException(String msg) {
        super(msg);
    }
}

class Vote {
    static void checkAge(int age) throws InvalidAgeException {
        if (age < 18)
            throw new InvalidAgeException("Not eligible to vote");
        else
            System.out.println("Eligible to vote");
    }

    public static void main(String[] args) {
        try{
            int age = 16;
            checkAge(age);
        }
        catch(InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output

```
Not eligible to vote
```

Concepts Used

- InvalidAgeException – a user-defined exception created by extending Exception.
- throw – explicitly throws the exception when age is below 18.
- throws – declares that the method may throw InvalidAgeException.
- try-catch – handles the user-defined exception.

3. Handling Multiple Built-in Exceptions

Question: Write a Java program to demonstrate the handling of `ArithmeticException`, `ArrayIndexOutOfBoundsException`, and `NumberFormatException` using appropriate catch blocks.

Program

```
class ExceptionDemo {
    public static void main(String[] args) {

        try{
            int a= 10, b = 0;
            System.out.println(a / b);
        }
        catch(ArithmeticException e) {
            System.out.println("Arithmetic Exception");
        }

        try{
            int arr[] = {1, 2, 3};
            System.out.println(arr[5]);
        }
        catch(ArrayIndexOutOfBoundsException e) {
            System.out.println("Array Index Exception");
        }

        try{
            int n = Integer.parseInt("abc");
            System.out.println(n);
        }
        catch(NumberFormatException e) {
            System.out.println("Number Format Exception");
        }
    }
}
```

Output

```
Arithmetic Exception
Array Index Exception
Number Format Exception
```

Concepts Used

- `ArithmeticException` – occurs during invalid arithmetic operations.
- `ArrayIndexOutOfBoundsException` – occurs when an invalid array index is accessed.
- `NumberFormatException` – occurs when a string cannot be converted into a valid number.
- Separate catch blocks handle each exception appropriately.

4. Two Threads: Numbers and Squares

Question: Write a Java program to create two threads using the Thread class. One thread should display numbers from 1 to 10, while the other displays their squares.

Program

```
class NumberThread extends Thread {
    public void run() {
        for(int i = 1; i <= 10; i++)
            System.out.println("Number: " + i);
    }
}

class SquareThread extends Thread {
    public void run() {
        for(int i = 1; i <= 10; i++)
            System.out.println("Square: " + (i * i));
    }
}

class Main {
    public static void main(String[] args) {
        NumberThread t1 = new NumberThread();
        SquareThread t2 = new SquareThread();

        t1.start();
        t2.start();
    }
}
```

Output

Note: Thread execution order may vary depending on scheduling.

```
Number: 1
Square: 1
Number: 2
Square: 4
Number: 3
Square: 9
Number: 4
Square: 16
Number: 5
Square: 25
Number: 6
Square: 36
Number: 7
Square: 49
Number: 8
Square: 64
Number: 9
Square: 81
Number: 10
Square: 100
```

Concepts Used

- Thread class – used to create and define separate threads.
- run() – contains the task performed by each thread.
- start() – starts the execution of a thread.
- Multithreading – allows both tasks to execute concurrently.

5. Multiple Threads using Runnable

Question: Write a Java program to create multiple threads by implementing the Runnable interface. Display the name and ID of each thread during execution.

Program

```
class MyThread implements Runnable {
    public void run() {
        Thread t = Thread.currentThread();
        System.out.println("Thread Name: " + t.getName());
        System.out.println("Thread ID: " + t.getId());
    }
}

class Main {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyThread(), "Thread-1");
        Thread t2 = new Thread(new MyThread(), "Thread-2");
        Thread t3 = new Thread(new MyThread(), "Thread-3");

        t1.start();
        t2.start();
        t3.start();
    }
}
```

Output

Note: Thread execution order may vary depending on scheduling.

```
Thread Name: Thread-1
Thread ID: 21
Thread Name: Thread-2
Thread ID: 22
Thread Name: Thread-3
Thread ID: 23
```

Concepts Used

- Runnable interface – used to define the task performed by each thread.
- run() – contains the code executed by the thread.
- getName() – returns the name of the current thread.
- getId() – returns the unique ID of the current thread.
- start() – starts each thread.

6. Threads with Different Priorities

Question: Write a Java program to create three threads with different priorities and display their priority values along with their execution details.

Program

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread Name: " + getName());
        System.out.println("Thread ID: " + getId());
        System.out.println("Priority: " + getPriority());
        System.out.println("Thread is running\n");
    }
}

class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        MyThread t3 = new MyThread();

        t1.setName("Low Priority");
        t2.setName("Normal Priority");
        t3.setName("High Priority");

        t1.setPriority(Thread.MIN_PRIORITY);
        t2.setPriority(Thread.NORM_PRIORITY);
        t3.setPriority(Thread.MAX_PRIORITY);

        t1.start();
        t2.start();
        t3.start();
    }
}
```

Output

Note: Thread execution order may vary depending on scheduling.

```
Thread Name: Low Priority
Thread ID: 21
Priority: 1
Thread is running

Thread Name: Normal Priority
Thread ID: 22
Priority: 5
Thread is running

Thread Name: High Priority
Thread ID: 23
Priority: 10
Thread is running
```

Concepts Used

- Thread priority – assigns a priority level to a thread.
- MIN_PRIORITY – value 1.
- NORM_PRIORITY – value 5.
- MAX_PRIORITY – value 10.
- setPriority() and getPriority() set and display priority values.

7. Synchronization with Bank Account

Question: Write a Java program to demonstrate synchronization by allowing two threads to access and update a shared bank account safely.

Program

```
class BankAccount {
    private int balance = 1000;

    synchronized void withdraw(int amount) {
        if (balance >= amount) {
            System.out.println(Thread.currentThread().getName()
                + " withdrawing " + amount);
            balance -= amount;
            System.out.println("Balance: " + balance);
        } else {
            System.out.println("Insufficient balance");
        }
    }
}

class MyThread extends Thread {
    BankAccount account;

    MyThread(BankAccount account) {
        this.account = account;
    }

    public void run() {
        account.withdraw(600);
    }
}

class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        MyThread t1 = new MyThread(account);
        MyThread t2 = new MyThread(account);

        t1.setName("Thread-1");
        t2.setName("Thread-2");

        t1.start();
        t2.start();
    }
}
```

Output

Note: Thread execution order may vary depending on scheduling.

```
Thread-1 withdrawing 600
Balance: 400
Insufficient balance
```

Concepts Used

- Synchronization – prevents multiple threads from modifying shared data at the same time.
- synchronized – allows only one thread to execute the withdraw method at a time.
- Shared resource – both threads access the same BankAccount object.
- Threads – two threads attempt to withdraw from the shared account.

8. Producer-Consumer using wait() and notify()

Question: Write a Java program to implement a simple producer-consumer problem using wait() and notify().

Program

```
class Buffer {
    int value;
    boolean available = false;

    synchronized void produce(int value) throws InterruptedException {
        while(available)
            wait();

        this.value = value;
        available = true;
        System.out.println("Produced: " + value);
        notify();
    }

    synchronized void consume() throws InterruptedException {
        while (!available)
            wait();

        System.out.println("Consumed: " + value);
        available = false;
        notify();
    }
}

class Producer extends Thread {
    Buffer b;
    Producer(Buffer b) { this.b = b; }

    public void run() {
        try{
            for (int i = 1; i <= 5; i++)
                b.produce(i);
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

class Consumer extends Thread {
    Buffer b;
    Consumer(Buffer b) { this.b = b; }

    public void run() {
        try{
            for (int i = 1; i <= 5; i++)
                b.consume();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

class Main {
    public static void main(String[] args) {
        Buffer b = new Buffer();
        new Producer(b).start();
        new Consumer(b).start();
    }
}
```

Output

Note: Thread execution order may vary depending on scheduling.

```
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
```

Concepts Used

- Producer – produces values in the shared buffer.
- Consumer – consumes values from the shared buffer.
- wait() – makes a thread wait until the required condition is satisfied.
- notify() – wakes a waiting thread after the buffer state changes.
- synchronized – provides safe access to the shared buffer.

9. Three Threads with Different Tasks

Question: Write a Java program to create three threads that perform three different tasks simultaneously: displaying numbers, calculating the sum of numbers, and displaying even numbers.

Program

```
class NumberThread extends Thread {
    public void run() {
        for(int i = 1; i <= 10; i++)
            System.out.println("Number: " + i);
    }
}

class SumThread extends Thread {
    public void run() {
        intsum = 0;
        for(int i = 1; i <= 10; i++)
            sum += i;
        System.out.println("Sum: " + sum);
    }
}

class EvenThread extends Thread {
    public void run() {
        for(int i = 1; i <= 10; i++)
            if (i % 2 == 0)
                System.out.println("Even: " + i);
    }
}

class Main {
    public static void main(String[] args) {
        new NumberThread().start();
        new SumThread().start();
        new EvenThread().start();
    }
}
```

Output

Note: Thread execution order may vary depending on scheduling.

```
Number: 1
Number: 2
Even: 2
Number: 3
Even: 4
Sum: 55
Number: 4
Number: 5
Even: 6
Number: 6
Even: 8
Number: 7
Number: 8
Even: 10
Number: 9
Number: 10
```

Concepts Used

- Multithreading – allows multiple tasks to execute concurrently.
- Thread class – used to create three separate threads.
- run() – defines the task performed by each thread.
- start() – starts each thread's execution.
- The three tasks display numbers, calculate the sum, and display even numbers.

10. Safe Thread Suspension, Resumption and Termination

Question: Write a Java program to demonstrate safe thread suspension, resumption, and termination using a control flag and `interrupt()` instead of the deprecated `suspend()`, `resume()`, and `stop()` methods.

Program

```
class Worker extends Thread {
    private volatile boolean running = true;
    private volatile boolean suspended = false;

    public void run() {
        try{
            int i = 1;
            while(running){
                synchronized(this) {
                    while (suspended)
                        wait();

                    System.out.println("Working: " + i++);
                    Thread.sleep(500);
                }
            } catch (InterruptedException e) {
                System.out.println("Thread interrupted");
            }

            System.out.println("Thread terminated");
        }

        synchronized void suspendThread() {
            suspended = true;
        }

        synchronized void resumeThread() {
            suspended = false;
            notify();
        }

        void stopThread() {
            running = false;
            interrupt();
        }
    }

    class Main {
        public static void main(String[] args) throws Exception {
            Worker w = new Worker();
            w.start();

            Thread.sleep(1500);
            w.suspendThread();
            System.out.println("Thread Suspended");

            Thread.sleep(1500);
            w.resumeThread();
            System.out.println("Thread Resumed");

            Thread.sleep(1500);
            w.stopThread();
            System.out.println("Thread Stopping");
        }
    }
}
```

Output

Note: Thread execution order may vary depending on scheduling.

```
Working:          1
Working:          2
Working:  3
Thread    Suspended
Thread    Resumed
Working:          4
Working:          5
Working:  6
Thread    Stopping
Thread interrupted
Thread terminated
```

Concepts Used

- Control flag – volatile flags safely control suspension and termination.
- wait() and notify() – safely pause and resume the thread.
- interrupt() – wakes a sleeping thread so it can terminate.
- Deprecated suspend(), resume(), and stop() methods are avoided.
- volatile – ensures changes to control flags are visible between threads.